The Skyline Problem

Vizualizacija i implementacija algoritma za određivanje panorame grada

Matematički fakultet, Univerzitet u Beogradu

Nemanja Ršumović 1016/2025

Sažetak

U ovom radu opisan je projekat koji rešava klasičan algoritamski problem poznat kao *The Skyline Problem* kojim se određuje panorama grada na osnovu skupa pravougaonih zgrada zadatih svojim koordinatama i visinama. Projekat je implementiran u programskom jeziku C++, korišćenjem Qt biblioteke za grafički korisnički interfejs. Rešenje se zasniva na pristupu *podeli pa vladaj* (eng. *divide and conquer*), čime se problem rešava u vremenskoj složenosti $O(n \log n)$. U radu su opisani problem, svrha projekta, algoritam i njegova analiza složenosti, arhitektura implementacije, grafički interfejs aplikacije, kao i specijalni slučajevi koje rešenje mora ispravno da obradi.

Sadržaj

1 Uvod **1**

2 Opis problema **2**

3 Algoritam i analiza složenosti **2**

3.1 Pristup podeli pa vladaj 2

3.2 Analiza vremenske i prostorne složenosti 2

4 Arhitektura i implementacija **3**

5 Grafički korisnički interfejs **3**

6 Specijalni slučajevi **4**

7 Zaključak **5**

Literatura **5**

1 Uvod

Određivanje panorame grada je jedan od klasičnih problema iz oblasti algoritama i struktura podataka, koji se često sreće na platformama za vežbanje algoritamskog razmišljanja poput LeetCode-a [1]. Problem kombinuje elemente geometrije, sortiranja i pristupa *podeli pa vladaj*.

Projekat za cilj ima implementaciju efikasnog algoritma koji za zadati skup zgrada izračunava tačke koje jednoznačno opisuju siluetu grada posmatranu iz daljine, kao i pravljenje desktop aplikacije sa grafičkim korisničkim interfejsom koja korisniku omogućava da učita proizvoljan skup zgrada, vizuelno ih prikaže, pokrene proračun i na kraju vidi izračunatu panoramu prikazanu u vidu stilizovane noćne scene grada. Ovim se demonstrira praktična primena pristupa *podeli pa vladaj*, kao i povezivanje algoritamskog rešenja sa vizuelnim prikazom rezultata kroz Qt frejmvork.

2 Opis problema

Problem 1 *(eng. The Skyline Problem). Data je gradska panorama posmatrana iz daljine, sastavljena od n pravougaonih zgrada. Svaka zgrada i zadata je uređenom trojkom celih brojeva (L_i, R_i, H_i), gde L_i i R_i predstavljaju x-koordinate leve i desne ivice zgrade ($L_i < R_i$), a $H_i > 0$ njenu visinu. Sve zgrade se oslanjaju na zajedničku osnovu (nivo tla $y = 0$). Potrebno je odrediti **panoramu**: skup tačaka koje jednoznačno opisuju spoljnu konturu (siluetu) svih zgrada posmatranih zajedno, isključujući sve unutrašnje linije koje nisu deo spoljašnjeg obrisa.*

Izlaz algoritma predstavlja se kao lista *ključnih tačaka* (eng. *key points*) oblika (x, y), poređanih po rastućoj x-koordinati, pri čemu svaka tačka označava mesto na kome se visina panorame menja. Poslednja tačka uvek ima visinu 0.

Primer 2.1 *Za skup zgrada*

$$\texttt{buildings} = [[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]$$

panorama je data listom ključnih tačaka

$$[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]].$$

Broj ključnih tačaka izlazne panorame u opštem slučaju **nije** jednak broju zgrada. Na primer, susedne zgrade jednake visine spajaju se u jednu konturu, dok visoke zgrade mogu u potpunosti prekriti niže (videti sekciju 6).

3 Algoritam i analiza složenosti

3.1 Pristup podeli pa vladaj

Rešenje problema zasniva se na pristupu *podeli pa vladaj*, analognom algoritmu *merge sort*. Ako ulazni skup sadrži samo jednu zgradu, njena panorama je trivijalna: sastoji se od tačke na levoj ivici sa visinom zgrade i tačke na desnoj ivici sa visinom 0. Ako sadrži više zgrada, on se rekurzivno deli na dve otprilike jednake polovine (funkcija `divide` u fajlu `skyline.cpp`), panorama se izračunava za svaku polovinu ponaosob, a zatim se dve dobijene panorame **spajaju** (*merge*) u jedinstvenu panoramu.

Ključni deo algoritma je funkcija `merge`, koja dve već izračunate panorame (levu i desnu) spaja u jednu. Postupak je sličan spajanju dva sortirana niza u algoritmu *merge sort* gde se kroz obe panorame prolazi istovremeno po rastućoj x-koordinati, a na svakoj tački se računa trenutna visina kao maksimum trenutnih visina leve i desne panorame. Nova ključna tačka se dodaje u izlaz samo ako se izračunata visina razlikuje od prethodno dodate, čime se izbegavaju redundantne tačke.

3.2 Analiza vremenske i prostorne složenosti

Neka je n broj zgrada u ulaznom skupu. Rekurzija koja opisuje vreme izvršavanja algoritma odgovara rekurziji algoritma *merge sort*:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

gde je $O(n)$ vreme potrebno za `merge` korak na datom nivou rekurzije. Po *master teoremi*, rešenje ove rekurzije je $T(n) = O(n \log n)$. Dubina rekurzije iznosi $O(\log n)$, a ukupan rad svih poziva `merge` funkcije na svakom nivou iznosi $O(n)$. Prostorna složenost iznosi $O(n)$.

Naivno rešenje bi redom prolazilo kroz sve x-koordinate od najmanje do najveće i za svaku od njih proveravalo koja zgrada je na tom mestu najviša. Ovakav pristup ima vremensku složenost $O(n \cdot \max(R_i))$ u najgorem slučaju, dok algoritam zasnovan na podeli i vladaj postiže vremensku složenost $O(n \log n)$.

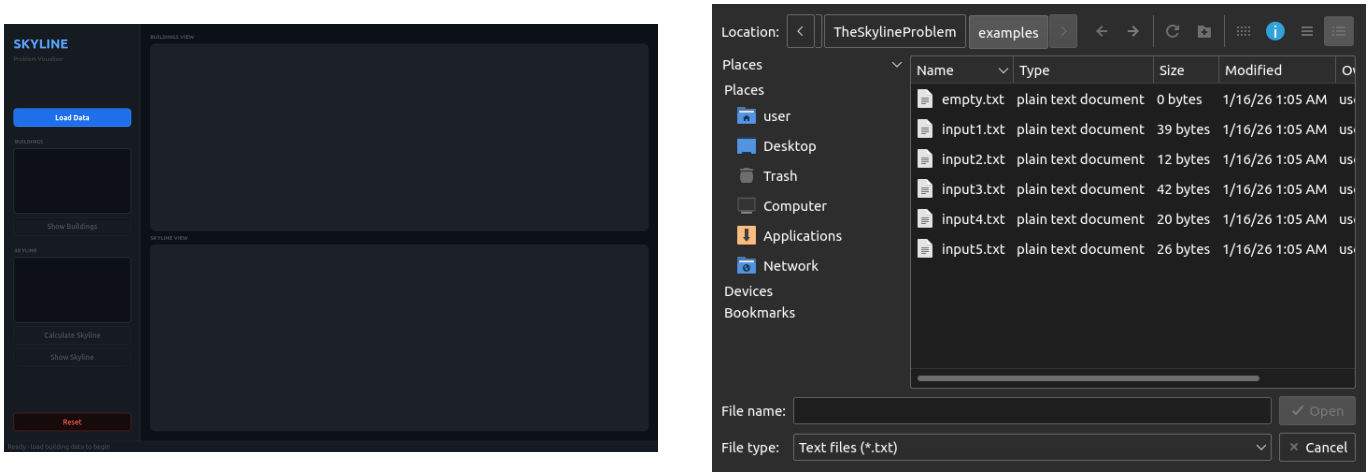
4 Arhitektura i implementacija

Projekat je implementiran u jeziku C++ (standard C++11), uz korišćenje Qt 5 frejmvorka. Kod je organizovan u dve logičke celine, klasu `Skyline` i klasu `MainWindow`. Klasa `Skyline` (`skyline.h/.cpp`) sadrži isključivo algoritamsku logiku nezavisnu od interfejsa sa jednom javnom metodom `getSkyline`. Klasa `MainWindow` (`mainwindow.h/.cpp/.ui`) sadrži logiku grafičkog interfejsa, tu se vrši učitavanje podataka iz fajla, prikaz zgrada i panorame na `QGraphicsScene`, kao i crtanje pozadinskih elemenata (noćno nebo, zvezde, mesec, put). Ovakvom podelom algoritam se može nezavisno testirati i menjati bez uticaja na interfejs. Ulazni podaci učitavaju se iz tekstualnog fajla, gde svaka linija sadrži tri cela broja razdvojena razmakom: L, R i H.

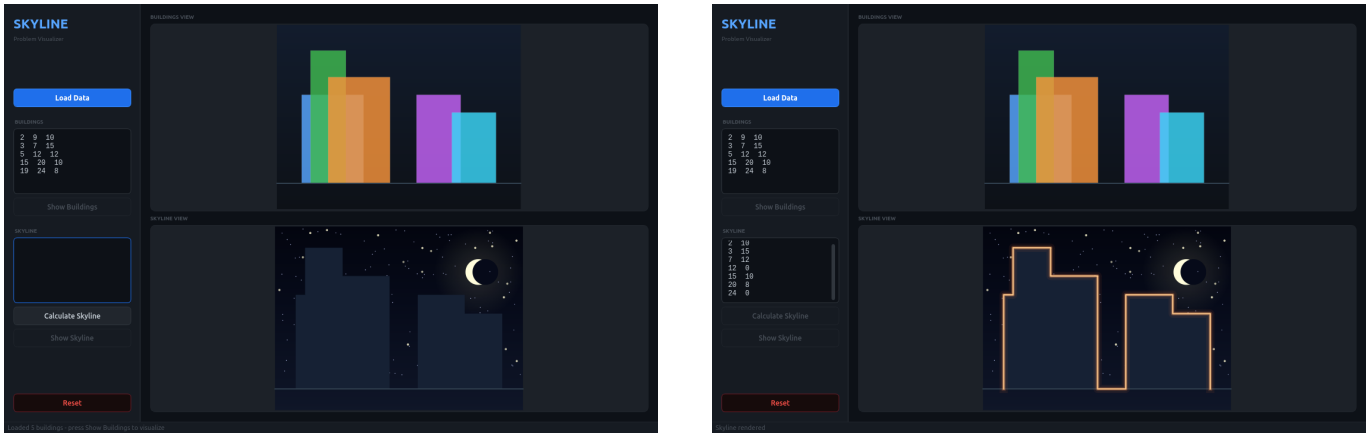
5 Grafički korisnički interfejs

Aplikacija ima tamnu (*dark mode*) temu i podeljena je na bočnu traku sa komandama i tekstualnim prikazom podataka, i dva grafička platna od kojih jedno za prikaz ulaznih zgrada (*Buildings View*), a drugo za prikaz izračunate panorame u vidu noćne scene grada (*Skyline View*).

Tok rada je sledeći: **Load Data** učitava skup zgrada iz tekstualnog fajla, **Show Buildings** vizuelno prikazuje učitane zgrade, **Calculate Skyline** pokreće algoritam nad učitanim podacima, **Show Skyline** prikazuje izračunatu panoramu kao svetleću konturu, a **Reset** vraća aplikaciju u početno stanje.



Slika 1: Početni izgled aplikacije i dijalog za učitavanje skupa zgrada iz tekstualnog fajla



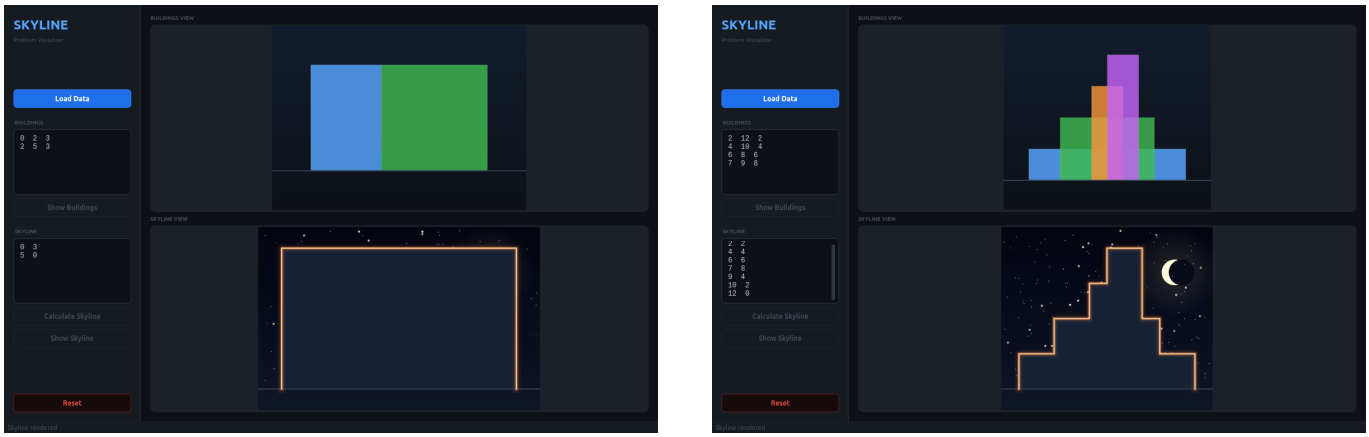
Slika 2: Prikaz učitanih zgrada (levo) i konačan izgled aplikacije nakon izračunavanja i prikaza panorame (desno)

6 Specijalni slučajevi

Prilikom implementacije i testiranja algoritma posebno su razmotreni sledeći granični slučajevi. Uz projekat je priložen i skup od pet test fajlova (direktorijum `examples/`), pomenutih uz odgovarajući slučaj.

Susedne zgrade jednake visine. Ako se dve zgrade nalaze neposredno jedna do druge i imaju jednaku visinu, njihova zajednička panorama treba da bude **jedna** neprekidna kontura, a ne dve odvojene konture sa suvišnom tačkom na mestu dodira. Ovaj slučaj je obrađen uslovom `peak != prevH` u `merge` funkciji, koji novu tačku dodaje u izlaz isključivo kada se visina panorame zaista promeni. Za ulaz `buildings` $= [[0, 2, 3], [2, 5, 3]]$ (`input2.txt`) očekivana panorama je $[[0, 3], [5, 0]]$, samo dve tačke za date dve zgrade (slika 3, levo).

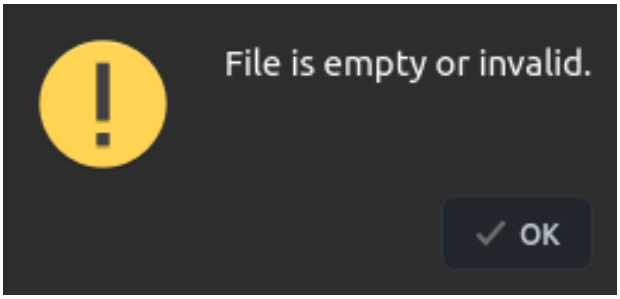
Potpuno ugnježđene (preklopljene) zgrade. Kada je jedna zgrada u potpunosti sadržana unutar druge, niže zgrade ne smeju uticati na konačnu panoramu na delu gde su prekrivene višom zgradom. Ovo se prirodno rešava funkcijom `std::max(hL, hR)` u `merge` koraku. Za skup od četiri zgrade koje se stepenasto preklapaju (`input5.txt`) dobijena panorama ima oblik piramide (slika 3, desno), gde svaka viša zgrada prekriva deo niže, tako da se u rezultatu vide sve promene visine, uključujući i kratkotrajni vrh najviše zgrade.



Slika 3: Susedne zgrade jednake visine (levo) i potpuno ugnježđene zgrade, panorama u obliku piramide (desno)

Nepovezane (razdvojene) zgrade. Ako između dve zgrade postoji praznina, panorama mora da se spusti na visinu 0 između njih, a ne da ostane na visini prethodne zgrade (`input4.txt`). Ovo se ostvaruje jer se prazan prostor tretira kao deo panorame sa visinom 0, pa se pri promeni sa visine 0 na visinu sledeće zgrade dodaje nova ključna tačka.

Prazan ili nevalidan ulazni fajl. Na nivou grafičkog interfejsa, obrađen je i slučaj kada korisnik pokuša da učita prazan fajl (`empty.txt`). Aplikacija u tom slučaju ne pokreće proračun, već korisniku prikazuje upozorenje (slika 4) i ostaje u prethodnom stanju.



Slika 4: Upozorenje prikazano korisniku prilikom učitavanja praznog fajla

7 Zaključak

U ovom radu opisan je projekat koji rešava *The Skyline Problem*, određivanje panorame grada na osnovu skupa pravougaonih zgrada. Implementirano rešenje zasniva se na pristupu *podeli pa vladaj*, analognom algoritmu *merge sort*, čime je postignuta vremenska složenost $O(n \log n)$. Pored algoritamskog dela, projekat obuhvata i grafički korisnički interfejs razvijen u Qt frejmvorku, koji korisniku omogućava da interaktivno učita proizvoljan skup zgrada i vizuelno prati sve korake, od prikaza ulaznih zgrada do konačnog prikaza panorame. Posebna pažnja posvećena je i graničnim slučajevima (susedne zgrade jednake visine, ugnježdene i razdvojene zgrade, nevalidan ulaz), čija je ispravna obrada neophodna za potpunu korektnost rešenja.

Literatura

[1] LeetCode. 218. The Skyline Problem. https://leetcode.com/problems/the-skyline-problem/, 2026. Datum pristupa: 12.7.2026.